OPERATIONS / ENGLISH EDITION

VAS

Deployment guide

Guided installation, configuration and release updates

DEPLOYMENT ENTRY POINT

```
./deploy.sh
```

PDF edition of **Docs/GUIDED_INSTALLATION.md**.

Follow the preparation and configuration steps before executing deployment commands. Long commands wrap for readability; copy them from the Markdown source when needed.

Internal links and the contents navigate this PDF. Repository-file links open the reference list at the end; companion files remain in the checkout.

14 October 2026
Installation · Configuration · Verification · Updates

Contents

1. Open the correct checkout 3
2. Choose one installation method 3
3. Answer the eight setup steps 4
Files to bring 5
Server configuration 6
GPU setup, in order 6
Default configuration 8
Connections configured automatically 8
4. Finish installation and verify it 9
Updating VAS 10
Command reference 11
Troubleshooting 12
Verification scope of this guide 13
Automation 13
Installer validation 14
Where files are saved 14
Repository references 17

The PDF bookmarks include subsections. Page numbers refer to this PDF edition.

VAS deployment guide

Entry point: `deploy.sh` in the VAS checkout. Reviewed against the scripts on 14 October 2026. This guide covers VAS installation, configuration, startup and updates. VMS has its own `docker-start.sh` and separate deployment guide.

1. Open the correct checkout

Open a terminal on the deployment server. Change to the VAS release directory; `/opt/vas` below is an example, so replace it with your actual checkout path:

```
cd /opt/vas
./deploy.sh
```

If the copied file is not executable, use `bash deploy.sh`. Both forms launch the same guide without arguments. You need Bash, Python 3 and an account allowed to use sudo. The wizard requests administrator access before collecting choices. Have the server's assigned IPv4 address, approved release files and model weights ready. Mount any dedicated data disk before choosing its location.

For an existing installation, read Updating VAS first. Preserve its environment, certificates, model files and storage. Moving the same server to another network has a separate offline move procedure.

No arguments are required. Run it in an interactive terminal. On Linux, the script requests your sudo password when administrator access is needed, then opens the eight-step guide. Review the plan and choose **1 — Install now** to proceed through installation, service startup and health checks. Pressing Enter at the review chooses **Preview checks only**, so select 1 when you want to install.

`sudo ./deploy.sh` and `./deploy.sh wizard` also open the same guide. Without a terminal, a bare invocation stops with instructions rather than deploying silently. Use explicit subcommands for automation; `--help` lists them.

2. Choose one installation method

Choice	What happens	What you need
1 — Load saved Docker images	Imports archives, checks every image required by Compose, and starts with `--no-build --pull never`.	Matching VAS release checkout, saved images (or images already loaded), and model weights.
2 — Fresh build	Uses the existing host setup, configuration and build stages, then starts and checks the services.	VAS source checkout, model weights, and access to build dependencies; offline builds require those dependencies locally.

Both choices preserve existing volumes. On an existing deployment, the protected backup/upgrade flow is used. “Fresh build” never means deleting the database.

3. Answer the eight setup steps

The addresses and paths below are examples. Use your server's actual values.

Step / displayed prompt	What to enter	What to check before continuing
1 — Installation method, `Choose 1 or 2`	`1` for saved images; `2` to build this release.	The default is build. Use the release checkout matching the images.
2 — Installation files, `Release package directory (Enter to use files already here)`	Example `/media/vas-release`, or Enter when the files are already in place.	The directory exists and has the required weights/manifest.
Load mode only: `Image archive path (Enter when finished)`	Enter one full archive path per prompt, then Enter to finish.	Already loaded images can be reused; every required service tag must exist.
3 — Server network and browser address, `SERVER_IP`	The IPv4 address assigned to this server on the client network, for example `10.20.30.40`.	Use neither a camera's address nor `0.0.0.0`/`localhost`.
`VAS browser URL`	`https://10.20.30.40` or a DNS name pointing to that server.	HTTPS port 443 is supported; clients must route to the address. Existing installations keep their configured primary URL.
4 — Hardware and connectivity, `Hardware option`	`1` requires NVIDIA GPU; `2` selects CPU.	CPU is lower throughput. GPU selection does not certify a camera count.
GPU only: `GPU setup option`	`1` verifies an existing setup; `2` opts into installation/repair.	Driver installation may require a manual reboot; Docker runtime repair can interrupt other containers. See GPU setup.
`Installation connectivity`	`1` allows installation downloads; `2` requires offline prerequisites.	Runtime remains subject to the production offline policy in either case.
5 — Default configuration, `Configuration option`	`1` keeps displayed values; `2` customizes the four displayed settings.	Check log level, backup retention and available storage. Person-history and image retention are separate.
6 — Service connections, `Service configuration option`	`1` keeps values; `2` chooses model names / notebook URL.	Ollama models must fit the hardware. A notebook URL does not install Jupyter.
7 — Storage and model/map files, `Storage option`	`1` keeps storage; `2` chooses a host data directory for a new installation.	Existing volumes are preserved. For option 2, use an already mounted location such as `/srv/vas-data`.
8 — Review, `Continue`	Enter **`1` to install**, `2` to preview, or `3` to cancel.	Enter alone selects **preview**, so it does not deploy.

Enter file paths without surrounding quotes; spaces are supported. Press Enter to accept the displayed default. The review defaults to preview checks. Cancelling or closing input during the questions makes no installation changes.

After **Install now**, the installer runs one sequence:

1. Check the host and Docker prerequisites.
2. Prepare directories, secret files and HTTPS certificates.
3. Save configuration; check GPU allocation, weights, network and optional assets.
4. Load and verify all required images, or build the application images.
5. Prepare the database and preserve/back up existing data when applicable.
6. Start services in dependency order, with migrations gating application startup.
7. Check configured language models, run health checks, and print login/integration instructions.

Each stage names what it is doing and reports its result. A mandatory failure stops at that stage with the reason and log location. Fix that issue and rerun the guide; completed setup is reused. An existing installation follows the protected upgrade sequence, including a configuration snapshot before applying new guided settings.

Files to bring

Keep the release checkout with `deploy.sh`, `scripts/`, `docker/`, `alembic/`, and the other application files. An image archive alone is not an installation package.

Supply these verified weights in the checkout or under the package's `weights/`:

```
weights/det_10g.onnx
weights/w600k_r50.onnx
weights/WEIGHTS_MANIFEST.json
```

A package can have this layout:

```
vas-release/
  images/
    vas-images.tar
    dependencies.tar.gz
  weights/
    det_10g.onnx
    w600k_r50.onnx
    WEIGHTS_MANIFEST.json
```

Use archives made with **`docker save`**, not container filesystems made with `docker export`. Keep the exact release tags, including the application, worker, migration and supporting service images. The installer prints every missing tag and stops before database startup if the set is incomplete. Load mode does not silently build, pull, or retag a different image. Match CPU/GPU images to the hardware option and use the checkout shipped with that release.

Docker images do **not** contain the production database, saved face photos, Ollama model volume or other persistent data. Moving an existing installation requires its data backups as well. This guide installs VAS; VMS and separately managed chatbot services retain their own deployment procedures.

Server configuration

The guide delegates configuration to the existing deployment stages:

- Docker/Compose prerequisites and GPU allocation checks.
- Generated secrets, preserving existing passwords.
- HTTPS certificates for the selected DNS name or IPv4 address.
- `docker/.env`, production offline policy and migration revision.
- Weight verification, image preparation, database initialization/migrations, service startup and health/acceptance checks.

Python 3 is required to run the guide. GPU prerequisite installation is optional and configurable as described below. CPU mode has lower throughput. A fully offline installation also needs host packages, all Docker images and model data supplied locally. The built-in checks stop with instructions when a prerequisite is missing.

The installer records `SERVER_IP`, `PUBLIC_ORIGIN` and `PUBLIC_ORIGINS` in `docker/.env`. It adds `https://<SERVER_IP>` to the explicit allowed browser origins while preserving existing entries. The certificate must cover **every** allowed origin; a new certificate includes the chosen primary hostname and server IP. An existing certificate missing the new IP is reported and never silently replaced. If you provision a certificate yourself, include every hostname/IP in its SANs.

Use the server address clients can actually reach. The guide supports HTTPS on port 443; it does not assign an operating-system IP, change DNS or open firewall ports. Existing installations keep their configured URL in the guide. Address changes require a separate certificate/origin update, preserving any configured additional origins. Existing certificates are validated and never overwritten.

Internet availability in the guide concerns installation dependencies. Production application runtime retains the existing offline policy. Missing optional LLM models are reported separately; a Docker image import does not populate them.

GPU setup, in order

At hardware step 4, choose **NVIDIA GPU required** or **CPU only**. Choosing GPU means a missing driver cannot silently turn the deployment into a CPU installation. Then select one GPU setup policy:

Policy	Behavior
Verify existing (default)	Checks the driver and Docker NVIDIA runtime without changing them. Best for an already working server.
Install/repair	Preserves a compatible driver; installs a missing/incompatible driver on supported Ubuntu x86_64 hosts, or installs/configures the Container Toolkit when needed.

With install/repair, choose **auto** for Ubuntu's hardware-recommended driver, or enter an explicit package such as `nvidia-driver-580-open`. This is an example, not a universal package recommendation. The package must be reported as supported by `ubuntu-drivers devices`,

available from the configured repositories, and meet the image's driver policy. Unsupported cards/packages stop with instructions. The installer installs matching kernel headers and never disables Secure Boot.

The steps are:

1. Read the CUDA version from `docker/Dockerfile.gpu` and inspect the active driver.
2. Keep a compatible driver. If repair is requested and needed, select/check the Ubuntu package and install it with the matching kernel headers.
3. **Stop for a manual reboot after a driver installation.** Exit code `75` and `REBOOT_REQUIRED` mean deployment is incomplete. Save your work, reboot, complete MOK enrollment if prompted, then rerun the guide and choose NVIDIA GPU again. Rerunning before reboot will not reinstall the driver. A driver still broken after reboot stops for diagnosis rather than entering a reinstall loop.
4. Check/install NVIDIA Container Toolkit and configure Docker's NVIDIA runtime. Runtime repair restarts Docker and can interrupt other containers; this is stated before the final installation review. An existing working runtime is kept.
5. Start the application and require real SCRFD/ArcFace inference on CUDA. The `gpu-test` command uses the application container, without pulling another CUDA image. A driver version or `nvidia-smi` alone is not acceptance.

CUDA and cuDNN remain in the application image; the host does not need a separate CUDA compiler toolkit. The current image is CUDA **12.4.1**. This installer's conservative Linux driver baseline is **550.54.15**, matching CUDA 12.4 Update 1's corresponding driver release. NVIDIA also documents older-driver minor-version compatibility with restrictions; this installer does not assume that path. Newer compatible drivers are retained. A changed CUDA base image requires a review of this policy. See NVIDIA's CUDA 12.4.1 release notes.

The version baseline does not certify GPU architecture support, sufficient VRAM, or camera throughput. Actual application inference remains the required test.

Automatic kernel-driver installation is limited to **native Ubuntu x86_64** with network access. Other operating systems and fully offline hosts receive manual prerequisite instructions. WSL must use the Windows-side NVIDIA driver and Docker Desktop integration; the installer refuses a Linux kernel-driver installation there. Container Toolkit follows NVIDIA's installation procedure. For signing/enrollment, follow Ubuntu's Secure Boot guidance.

Automation examples:

```
# Require GPU, checking existing prerequisites only:
sudo ./deploy.sh deploy --gpu --gpu-setup=verify --yes \
  --server-ip=10.20.30.40 --public-origin=https://10.20.30.40

# Opt into missing-driver/runtime installation or repair:
sudo ./deploy.sh deploy --gpu --gpu-setup=install --nvidia-driver=auto --yes \
  --server-ip=10.20.30.40 --public-origin=https://10.20.30.40
```

`--dry-run` and `validate` never install drivers or restart Docker. If a prerequisite is missing, they explain what must be installed and stop before dependent GPU checks. No automatic reboot is performed.

Default configuration

The guide displays existing values from `docker/.env`, or these defaults for a new installation. Press Enter to keep them, or select **Customize**:

Setting	New installation default	Meaning
`LOG_LEVEL`	`INFO`	Normal operational logging; `WARNING` and `ERROR` are quieter.
`DATA_RETENTION_DAYS`	`365`	Age-based detection cleanup only when `PRESERVE_PERSON_HISTORY=false`; generated reports also use this period.
`BACKUP_RETENTION_DAYS`	`14`	Older automatic backups become eligible for deletion.
`MAX_STORAGE_GB`	`5000`	Storage capacity used for dashboard reporting; not a disk quota or reservation.

Person-history preservation defaults to **`PRESERVE_PERSON_HISTORY=true`**. Routine cleanup keeps recorded person history and embeddings; logs and temporary data still have their own expiry. Camera images use **`SNAPSHOT_RETENTION_DAYS`** (default 90), subject to retained references. These two settings are not among the four wizard prompts; review their effective values in **Settings** after login. See Person history and image retention.

Choose retention periods according to your storage and operational requirements. Reducing backup/image retention can remove eligible backups/images; turning history preservation off also enables the applicable historical cleanup. Storage reporting should reflect capacity available to this installation; the default does not mean the machine has 5 TB available.

Values are validated and included in the review before installation. Settings are written to `docker/.env` only during the deployment stages; a dry run prints the planned changes. Existing deployments snapshot their configuration before applying the guided choices. Keep this file private because it also contains credentials. Application settings previously saved through the Settings page may override these startup defaults. This step does not reset those settings or change recognition thresholds, GPU concurrency, camera FPS, or the database schema.

Connections configured automatically

PostgreSQL, Redis, Ollama and Martin use private Docker service names. Users do not need to enter database/cache IPs or database passwords. The existing secret generator and role setup supply them. HTTPS remains on port 443, webhook authentication stays required, and the migration revision is derived from the release checkout.

`OLLAMA_MODEL` and `OLLAMA_SQL_MODEL` default to the existing production value `gpt-oss:20b`. The guide preserves configured model names and offers customization; choose models that fit the target hardware. The model stage checks these actual configured values, not an unrelated manifest list. Missing models are fetched only when installation connectivity allows it; an offline host must have them preloaded.

`ML_NOTEBOOK_URL` stays empty until a notebook service is separately deployed. Use its HTTPS URL without embedded credentials or token query strings. Setting a URL does not install Jupyter. The current wizard requires an absolute HTTPS URL, such as `https://vas.example/notebooks/lab`; it will reject a saved relative path such as `/notebooks/lab` even when you choose to keep existing values. Preserve the same working service by supplying its full HTTPS URL; use `-` only when intentionally disabling the notebook link. Prepared `.mbtiles` archives and map fonts must also be supplied separately; the guide reports missing assets. The final runtime checks, not the existence of files, determine service health.

For VMS ingestion, configure the VMS publisher after installation:

```
URL: https://<VAS-server-IP-or-hostname>/api/webhook/<pipeline_id>
Header: X-Webhook-Key: <VAS ingest credential>
```

For the supplied VMS launcher, set `WEBHOOK_BASE_URL` to the HTTPS origin only (for example `https://vas.example`), and put its credential in `WEBHOOK_AUTH_TOKEN`. VMS constructs `/webhook/{pipeline_id}` and sends bearer authentication itself; do not paste the complete per-camera URL into its base-URL setting. The URL/header above describe a generic sender, not VMS field values.

Read the generated token privately from `secrets/webhook_api_keys`, or issue a credential in VAS. Trust the VAS public CA certificate on the VMS sender and verify an actual camera detection. The installer prints these directions but does not change VMS pipelines, create camera records, or expose the credential in its log.

4. Finish installation and verify it

The final report shows results and the deployment log location. The guide prints:

1. The public CA certificate to trust on client PCs: `certs/internal-ca.crt`.
2. The browser URL.
3. The initial admin password file: `secrets/bootstrap_admin_password`. Read it privately with `sudo`; passwords are not printed in the wizard or its log.
4. How to check status and logs, then add cameras and confirm a real detection.

New installations use the default `admin` account unless bootstrap configuration was customized. Existing accounts and passwords remain in effect. Never distribute `internal-ca.key` or other private keys to client PCs.

1. Wait for the final deployment report. Required stages must pass. A preview report means no installation was performed.
2. Trust **only** `certs/internal-ca.crt` on each client computer if using the generated CA. Keep CA/server private keys on the managed server.
3. Open the chosen URL, for example `https://10.20.30.40/signin`.

4. For a new installation, read the bootstrap password privately on the server:

```
sudo cat secrets/bootstrap_admin_password
```

5. Sign in as `admin` unless bootstrap configuration was customized, then complete any required password change. Existing deployments use their existing account passwords; the bootstrap file does not reset an existing password.
6. Check the services and acceptance report from the VAS checkout:

```
sudo ./deploy.sh status
sudo ./deploy.sh health
```

   The API and dependencies should be healthy. The `migrate` job exiting with code 0 is expected: it is a one-shot job, not a continuously running service. Required GPU acceptance includes actual model inference, not just `nvidia-smi`. For a separate GPU check, run `sudo ./deploy.sh gpu-test --gpu` after the API is running and require the successful SCRFD/ArcFace inference result. A warning that the API is stopped, or a skipped stage, does not prove GPU acceptance.
7. Connect one approved test camera using the **separate VMS guide** and the ingest credential instructions. Confirm the publisher accepts the webhook, then verify a new detection in Live Feeds and its expected pipeline. A healthy server alone does not prove the camera-to-VAS connection or search accuracy.

For diagnosis, follow the API log and stop viewing it with Ctrl+C:

```
sudo ./deploy.sh logs face_recognition
```

Ctrl+C ends the log viewer; it does not stop VAS. Fix a failing prerequisite and rerun the guide. Check the final result again; a dry run or an image build alone is not a successful deployment.

Updating VAS

Use this section for an existing server. Plan an interruption and preserve the installed configuration and persistent storage while replacing release source.

Check the installed configuration first. `deploy.sh` reads `docker/.env` and selects its production GPU/storage overlays. An installation previously started with another `--env-file`, project name or extra Compose override must retain those choices. Do not assume the wrapper reproduces a custom deployment. These read-only commands show the Compose files and optional explicit environment file recorded on the standard production API container:

```
sudo docker inspect --format '{{index .Config.Labels
"com.docker.compose.project.config_files"}}' face_detector_prod-face_recognition-1
sudo docker inspect --format '{{index .Config.Labels
"com.docker.compose.project.environment_file"}}' face_detector_prod-face_recognition-1
```

A blank environment-file label is possible; it is not proof of missing settings. If the recorded configuration differs from this guide, have the operator reconcile it with the actual installed environment before using the wrapper to update it. Do not copy production secrets into a guide or share full Compose output.

For an installation already managed by this wrapper:

1. Run `sudo ./deploy.sh status` and `sudo ./deploy.sh validate` from the installed checkout. Resolve configuration errors before changing the running service.
2. Install the approved release source while retaining `docker/.env`, `secrets/`, `certs/`, `weights/`, map data, deployment state and the selected data volumes. Keep a recoverable copy of the current release and a verified external backup; see Backup and recovery for actual restore limitations.
3. Run `./deploy.sh` and choose the new release's image-loading or build path, or use this command for a source build on an installation that requires GPU:

   ```
   sudo ./deploy.sh upgrade --gpu
   ```

   Use `--cpu` instead for an intentionally CPU-only installation; add `--offline` when installation downloads must remain disabled. Repeat the intended hardware and connectivity flags on maintenance commands: a bare `upgrade` can detect hardware/connectivity again rather than preserve a previous CLI choice. The guide asks you to confirm these choices explicitly.
4. Follow the backup, configuration snapshot, build/load, migration, rollout and acceptance stages. Verify the final result, login and one approved test capture.

The upgrade can migrate the schema when a release contains migrations. “Preserve volumes” does not mean “never changes database schema.” Image rollback is attempted on applicable upgrade failures only when the schema has not advanced. The current script has **no `rollback` subcommand**; schema changes or data-format changes need a release-specific recovery plan. Its backup/restore tooling has the limitations documented in the linked recovery guide.

Command reference

Run these from the VAS checkout. The primary installation command is still the bare `./deploy.sh`; helper scripts under `scripts/deploy/` are called by it.

Purpose	Command	Actual behavior
Interactive installation / guided update	`./deploy.sh`	Eight setup steps, review, installation and checks.
Read usage	`./deploy.sh --help`	Prints supported commands and options.
Validate prerequisites	`sudo ./deploy.sh validate`	Checks without installation changes; writes a diagnostic log.
Inspect status	`sudo ./deploy.sh status`	Lists recorded state and current services.
Acceptance checks	`sudo ./deploy.sh health`	Checks the installed stack without rebuilding it.
Storage locations	`sudo ./deploy.sh storage`	Prints persistent-data and model/map locations.

Purpose	Command	Actual behavior
Model file validation	`sudo ./deploy.sh model-check`	Checks required weights against the manifest.
GPU proof	`sudo ./deploy.sh gpu-test --gpu`	Requires GPU setup; the API must be running to execute the real inference check. A skipped/ warning-only result is not proof.
View API logs	`sudo ./deploy.sh logs face_recognition`	Follows recent logs; Ctrl+C exits the viewer.
Prepare host/config only	`sudo ./deploy.sh install`	Does not complete application build/start/acceptance.
Start through deployment stages	`sudo ./deploy.sh start`	Can build, initialize/check the database, back up and start services.
Reconcile/start existing stack	`sudo ./deploy.sh restart`	Uses Compose `up -d`; unchanged containers may stay running. It is not a forced process restart or a release rebuild.
Deploy new GPU release	`sudo ./deploy.sh upgrade --gpu`	Uses backup/snapshot/rollout checks; use `--cpu` for CPU-only, and add `--offline` when required.
Stop services	`sudo ./deploy.sh stop`	Stops this stack; preserves persistent volumes.
Create backup	`sudo ./deploy.sh backup`	Runs backup/checksum verification; follow the recovery guide to protect and rehearse restoration.

Routine deployment does not require `uninstall`; ordinary uninstall preserves volumes. `uninstall --purge-data` and `docker compose down -v` remove persistent volumes and are outside this guide.

Troubleshooting

What you see	What to do next
No installation happened	At review select **1 — Install now**. Enter selects preview.
`Permission denied` launching the file	Run `bash deploy.sh` from the VAS checkout; supply sudo credentials when prompted.
`The wizard requires a terminal`	Open an interactive SSH/local terminal, or use explicit automation flags below.

What you see	What to do next
Missing model or checksum mismatch	Supply the matching approved ONNX files and manifest; run `sudo ./deploy.sh model-check`. Do not rename models or regenerate a manifest just to bypass a failure.
Missing Docker image tag in load mode	Import the missing release archive or obtain the matching tag. `docker export` archives are unsuitable.
`REBOOT_REQUIRED` / exit 75	Save work, reboot manually, complete required driver enrollment and rerun the GPU setup.
GPU fails or falls back to CPU	Run `sudo ./deploy.sh gpu-test --gpu`; resolve the reported driver/toolkit/provider problem before accepting GPU deployment.
Login blocked by origin or certificate error	Verify the exact browser URL, `PUBLIC_ORIGINS`, certificate SANs, client CA trust and server reachability. Existing certificates are not silently replaced.
Notebook URL is rejected	Supply the existing service's full HTTPS URL, without tokens or embedded credentials. Relative paths are rejected by this wizard.
Storage layout mismatch	Remount the expected disk and keep the recorded `VAS_DATA_ROOT`. Do not substitute empty directories for existing data.
Migration/configuration failure	Keep the stage log from `logs/deploy/`; inspect `sudo ./deploy.sh logs migrate` or `logs face_recognition`. Correct the stated cause before rerunning.
Healthy VAS but no camera events	Check VMS publishing, the exact pipeline ID, ingest credential, TLS trust and network reachability; verify an approved capture end to end.

Verification scope of this guide

Commands, prompt labels and configuration paths were checked against the current repository and safe help output. This documentation update did not install packages, restart services or change databases. Installation on a new target still requires its own final health, GPU and camera acceptance checks; a camera-capacity claim requires a representative load test.

Automation

Build from source:

```
sudo ./deploy.sh deploy --yes --image-mode=build \
  --server-ip=10.20.30.40 --public-origin=https://vas.example
```

Load a release without building or pulling Docker images:

```
sudo ./deploy.sh deploy --yes --image-mode=load \
  --image-archive=/media/vas-images.tar \
  --deploy-package=/media/vas-release \
  --server-ip=10.20.30.40 --public-origin=https://10.20.30.40
```

Repeat `--image-archive` for additional archives. Add `--offline` when host/package network installation must also be disabled, or `--cpu` for CPU-only deployment. Use `--dry-run` first to inspect the plan.

Installer validation

```
bash deploy.sh --self-test
python3 -m unittest discover -s tests -p test_deploy_wizard.py -v
```

These use isolated fixtures and fake Docker calls; the certificate regression creates temporary certificates only. GPU tests replace package managers, driver probes and systemctl with stubs; they never install drivers or restart Docker. They verify installer behavior, not a full installation on a fresh server. Target-host service, GPU and camera checks remain part of deployment acceptance.

Where files are saved

Docker named volumes are production-appropriate persistent storage: they survive container replacement. They are directories on the Docker host, separate from the container writable layer. Moving to bind mounts does not itself improve durability or provide high availability. See Docker volumes.

Run this read-only report from the release checkout:

```
sudo ./deploy.sh storage
```

It lists the actual Docker volume locations, or their names before first startup, plus all model/map destinations. It does not display credential contents. Docker must be running and accessible. Default locations are inspected rather than assumed to be under `/var/lib/docker`; a daemon can use a different data directory.

What	Default storage / files to supply
PostgreSQL / Redis	`postgres_data` / `redis_data` volumes
Face photos, snapshots, application artifacts	`storage_data` volume
Local face indexes	`face_database_data` volume
Trained ML service models and manifests	`ml_artifacts_data` volume, shared by API and ML worker

What	Default storage / files to supply
Application logs	`logs_data` volume
Ollama models	`ollama_models` volume
Embedding model caches	`chromadb_cache` / `hf_cache_data` volumes
Automatic backups	`backup_data` volume
Metrics / Grafana	`prometheus_data` / `grafana_data` volumes
Optional vLLM / Milvus	`vllm_models`, `milvus_etcd`, `milvus_minio`, `milvus_data` volumes
Face detector ONNX	`<checkout>/weights/det_10g.onnx`
Face recognizer ONNX	`<checkout>/weights/w600k_r50.onnx`
Trusted weight checksums	`<checkout>/weights/WEIGHTS_MANIFEST.json`
Map archives	`<checkout>/map-data/production/*.mbtiles`
Map fonts	`<checkout>/map-data/production/fonts/*.ttf` or `*.otf`
Map metadata / verification results	`<checkout>/map-data/metadata/`
Host deployment backups / configuration snapshots	`<checkout>/backups/`
Installer logs / state	`<checkout>/logs/deploy/` and `<checkout>/.deployment/`
HTTPS / credentials / environment	`<checkout>/certs/`, `secrets/`, `docker/.env`

Volume names have the `face_detector_prod_` prefix. Supply both ONNX weights and the matching trusted manifest before installation, or use `--deploy-package` to import the release's `weights/`. Do not put the face detector weights into the ML artifact volume: trained tabular models and face recognition ONNX weights have different consumers. Maps and fonts are copied by the operator to the paths above; map content checks still determine whether each archive is usable.

Weights and map archives are mounted read-only into consumers; map metadata has a separate writable mount. These paths deliberately stay in the checkout in this release. Preserve them when replacing the checkout. A stable managed checkout is acceptable; relocating these assets needs corresponding mount and verification changes, not just moving the files.

Optional: choose a dedicated data disk before a fresh install

Mount the disk on the host first and ensure it is mounted at boot. In the guide's storage step choose option 2 and enter, for example, `/srv/vas-data`. The installer saves `VAS_DATA_ROOT='/srv/vas-data'` in `docker/.env` and includes `docker/docker-compose.prod.storage.yml`. It creates a directory for each volume:

```
/srv/vas-data/postgres_data/
/srv/vas-data/storage_data/
/srv/vas-data/ml_artifacts_data/
/srv/vas-data/backup_data/
... one directory for every volume in the table ...
```

These are still named Docker volumes, backed by your chosen host directories. Container paths and sharing between services stay the same. New directories receive service ownership; existing files are never moved or recursively re-owned. The installer refuses a changed layout for existing volumes, nonempty unregistered data directories, symlinks, and missing directories behind existing volumes. If a disk is missing, remount it; do not create empty replacements. Storage on a remote Docker daemon is not supported by this host-directory setup.

For automation, configure `VAS_DATA_ROOT` in `docker/.env` before first installation (or pass it as an environment variable). `deploy.sh` selects the overlay for all its Compose operations. If using Docker Compose directly, include the storage overlay explicitly, and the GPU overlay when applicable:

```
docker compose --project-directory docker   -f docker/docker-compose.prod.yml   -f docker/
docker-compose.prod.storage.yml config --quiet
```

Prefer `deploy.sh` for installation because direct Compose bypasses its path and existing-volume checks. Leave `VAS_DATA_ROOT` empty for the default layout. Never clear/change it on an existing custom-storage deployment to try a different disk. Relocation is a separate, backed-up migration with services stopped and a verified restore; this guide does not perform one.

This setting covers the declared production data volumes, not every byte Docker writes: image layers, build cache and container stdout logs remain in daemon storage. Checkout files and separately deployed Jupyter storage also remain at their own locations. Monitor both the data disk and Docker's disk. Keep protected backups on a separate disk/server and test restores: an on-disk volume or a backup on the same disk alone does not protect against disk loss.

Storage regression tests (temporary fixtures; no live volume migration):

```
python3 -m unittest discover -s tests -p test_deploy_storage.py -v
```

Repository references

Links to source files and companion Markdown guides point here so this PDF can travel independently. Open these paths in the matching VAS checkout.

1. **Repository file**
   `Docs/offline-deployment.md`
2. **Repository file**
   `Docs/PERSON_HISTORY_RETENTION.md`
3. **Repository file**
   `Docs/ingest-credentials.md`
4. **Repository file**
   `Docs/dashboard.md`
5. **Repository file**
   `Docs/pipelines.md`
6. **Repository file**
   `Docs/backup-and-recovery.md`